NyayaSetu — AI consumer-rights assistant for India

Kuku AI Generalist Assignment · Problem 2 (AI Lawyer) · Prototype: Streamlit app + repo with run steps, raw outputs and evaluation report. All results below come from DEMO MODE (deterministic rules; no live LLM API was connected).

1. Problem chosen and why

Problem 2, an AI lawyer for ordinary people in India, narrowed to one high-volume, low-value dispute: problems with things bought online (refund never came, damaged or defective goods, 'delivered' but not received, not as advertised, charged twice). These are frequent, rarely worth a lawyer's fee, and the law gives a concrete path (seller, then the platform's grievance officer, then the National Consumer Helpline, then the District Commission, with no fee up to ₹5 lakh). Depth on one area lets every legal statement be checked against official text, which matters more for a legal tool than breadth.

2. Target user and the job

A first-time online shopper, English or Hindi speaking, with no legal knowledge, who has a receipt-sized dispute and would otherwise give up. Job: turn a messy story into (a) an organised case, (b) an honest view of where they stand, (c) documents they can send today, (d) the next step.

3. Assumptions and scope

Covered: consumer purchases of goods over websites/apps under the Consumer Protection Act 2019 and E-Commerce Rules 2020. Not covered: banking, insurance, housing, employment, criminal fraud (flagged and redirected). Users are assumed to be buying for personal use unless they say otherwise (the app asks). Legal sources were re-verified on 8 Nov 2026; E-Commerce Amendment Rules 2026 take effect on 1 Jan 2027 and are shown as a dated notice. No paid API: the default is a deterministic demo mode.

4. Product and workflow (Streamlit, 7 pages)

(1) Describe the problem in your own words; the app extracts amount, dates ('three weeks ago' becomes a date), platform, seller, order ID, ticket, evidence, then asks only the missing questions for that dispute type, one at a time, all skippable. (2) Editable case file with timeline, financial loss, provenance of every field and a missing-information list. (3) Rights: issue cards labelled What the law requires / Possible argument / Procedure / Possible remedy / Notice, each with how it applies, evidence held vs needed, next action and the official source (URL, authority, date checked). (4) Four drafts in English or Hindi — seller email, platform grievance-officer complaint, NCH grievance, Consumer Commission preparation draft — editable, TXT/PDF, gaps marked [MISSING]. (5) Action plan with official websites. Plus a sources & limitations page. Aadhaar/PAN/card/OTP are masked if typed; case data stays in the session.

5. Architecture

Pipeline: intake.py (rules: classify, extract, choose the next question), case_builder.py (JSON case, timeline, flags: limitation, commercial purpose, harm), rights_analyzer.py (rules keyed to dispute type + facts) using legal_retrieval.py (30-entry verified KB, BM25 + intent boosts) and legal_validation.py, complaint_generator.py (templates) and action_planner.py; pdf_generator.py (fpdf2 + HarfBuzz, merged Noto font for Hindi). llm_provider.py offers Demo / Anthropic / any OpenAI-compatible endpoint, switched by one environment variable, so a stronger model can be plugged in without code changes. Retrieval and validation never depend on the model.

6. Where AI is used and where rules are used

Rules: extraction, classification, all legal reasoning, all drafts (templates quote the user's words; no model-written facts in legal documents). AI (only when a key is configured): fill empty extraction fields (accepted only if the value literally appears in the user's text), write the plain-language summary and answer the question box from retrieved sources. Human in the loop: the user confirms the case file and edits every draft; nothing is sent or filed by the app. Cost at volume: demo mode is free to run; with an LLM, about two short calls per case, and legal text is cached in the KB rather than generated.

7. How invented law is prevented

(i) A curated knowledge base: each provision with official URL, issuing authority, excerpt and verification date (India Code, Gazette, Department of Consumer Affairs, NCH, PIB, Lok Sabha answer; re-verified 8 Nov 2026); secondary-only items are shown as 'verify before relying' notices, never as law. (ii) The rules choose the law, not the model. (iii) A validator extracts every Section/Rule/धारा, case-law pattern (X v. Y, AIR, SCC, CPJ), other Act and [KB:ID] tag from any draft or AI text; anything not verified fails, and failing AI text is replaced by the rule-based text. It also blocks outcome guarantees and re-checks the user's own edits live. (iv) Over-claiming guards, e.g. the 30-day unfair-trade-practice clause is not asserted before 30 days or when the user wants a replacement; the refund claimed is the amount the user confirmed, never the price by default. (v) The KB has no case law, so none is cited.

8. Testing method and actual results

7 scenarios (5 required + 2 deliberately difficult: vague facts; resale purchase from March 2024) run through the real code path; raw outputs saved unedited. Automated: categories 7/7; expected fields 53/53; 165 citations in 32 drafts all matched the KB; refund amount correct in all 32 drafts and 7/7 claimed-amount conversations (full, partial, unclear then re-asked, skipped, Hindi, excess); fake-LLM injection test rejected a fabricated 'Section 72A' + invented case, another Act + promise, and an untagged answer, and accepted a grounded answer (4/4); retrieval probes top-1 13/15, top-3 15/15; UI journeys 23/23 steps. These are my own tests, so I also read every output: manual review found 13 errors the checks missed, including a template stating the defect appeared 'shortly after delivery' (not said by the user), the unfair-trade-practice clause argued for a replacement request, 'mobile repair shop' read as 'wants a repair', and a key fact dropped from the quoted story. All 13 are now fixed. A final pass found 10 more, including drafts claiming the full price when the user wanted part of it (now an explicit, confirmed amount) and an Anthropic request that current models would reject (fixed; checked with the real SDK against a local mock — no live call made). On 3 stories written afterwards, 2 were first classified

as 'unknown'.

Case	Facts	Law identified	Draft usable?	Main issue in final run
TC01 refund, damaged phone	10/10	Yes; 30-day UTP held 'not yet'	Yes	Unknown if refund was formally 'accepted'
TC02 non-delivery (Hindi)	10/10	Yes	Yes (HI + EN)	Purpose not asked; Hindi not native-reviewed
TC03 defective washer	10/10	Yes; UTP not argued (replacement)	Yes	Manufacturer not suggested as a party
TC04 misleading listing	9/9	Yes	Yes	Depends on proof of original listing
TC05 double UPI charge	10/10	Yes; RBI route as notice only	Yes	No complaint date, timing not computed
TC06 vague facts (difficult)	n/a	Partly (category guessed)	Skeleton only	13 gaps flagged; 12 skipped questions
TC07 resale + 2024 (difficult)	4/4	Yes: consumer status + limitation	Lawyer first	Partial claim ₹48,000 asked and used

9. Known limitations and failure cases

Keyword extraction is brittle (Hinglish, unusual phrasing, wrong-size items); dates attach to the nearest keyword. The refund claimed is one total (no per-item breakdown). Marketplace vs inventory seller is not determined; the manufacturer is never suggested as a party. Hindi text is not native- or lawyer-reviewed. No real LLM was connected (demo mode for all results). The Commission draft is a preparation aid, not a filing-ready pleading. Not deployed; runs locally with three commands.

10. How it scales

Users: stateless, session-only app; horizontal scaling behind any host; per case cost is zero in demo mode. Categories: a new dispute type = KB entries + an issue rule + a question flow; landlord deposits, salary dues and insurance are natural next modules, each needing its own verified sources. Languages: UI strings and keyword lists are tables; an LLM extractor would remove most per-language rules. Quality: the test harness, validator and KB verification log become the regression suite; a lawyer-review queue can sit before the Commission draft. Re-verify the KB on each legal change (next: 1 Jan 2027).

11. Tools used, and one place AI got it wrong

Claude (Anthropic, used in the Claude app) for legal-source research, code, tests and drafting; web search/fetch against official sites; Python, Streamlit, fpdf2 + HarfBuzz, fontTools, Playwright for screenshots. Where AI was wrong: writing from memory, Claude placed the E-Commerce Rules' refund duty at Rule 4(9) and the seller's take-back duty at Rule 6(2). Checking the Gazette text showed Rule 4(9) is about consent, the refund duty is Rule 4(10) and take-back is Rule 6(3). Its memory also lacked the 2026 E-Commerce amendments and the 2026 RBI Ombudsman scheme. Both were caught by verifying every entry against official text before it entered the knowledge base — the same reason the app itself never answers legal questions from model memory. A second full check found Claude had still paraphrased Rule 6(3) with words the Rules do not contain ('as per the description on the platform'), and had sent a parameter current Claude models reject.

NyayaSetu is an AI-assisted legal information and document-preparation tool. It does not replace a qualified advocate or constitute a lawyer-client relationship. Legal outcomes depend on facts and applicable law.